

Documentazione Simulatore

Raul Jose Luizaga Yujra



Università degli Studi di Bergamo

Gennaio 2025

Indice

1	Webots	2
1.1	Build	2
1.2	Setup	2
1.3	Creazione di un nuovo package ROS2	2
1.4	Creazione di un nuovo Mondo di simulazione	3
1.5	Metodo alternativo per apportare modifiche	4
1.6	Selezione del laser	4
1.7	Util	4
1.8	aruco_msgs	4
1.9	aurora_msgs	4
1.10	rover_msgs	5
1.11	webots_hospital_tiago	5
2	Descrizione funzioni Driver Webots Tiago	7
2.1	init components	7
2.2	init_tf_odom	8
2.3	init_laser	8
2.4	setup_robot_devices	10
2.5	setup_publishers_and_subscribers	11
2.6	init_messages	12
2.7	setup_laser_msg	12
2.8	setup_rover_msg	14
2.9	init_info_camera	15
2.10	init_tf_camera	16
2.11	take_data_from_Webots_node	18
2.12	__cmd_vel_callback	18
2.13	__cmd_vel_callback	19
2.14	Funzione step	19

1 Webots

1.1 Build

Entrare nel path `/home/serl/GitHub/STRESA/ROS` ed eseguire **build.sh**.

1.2 Setup

Il seguente comando serve ad inizializzare le variabili d'ambiente:

Entrare nel path `/home/serl/GitHub/STRESA/ROS` ed eseguire **setup.sh**.

1.3 Creazione di un nuovo package ROS2

- **Primo metodo:** Creazione del pacchetto utilizzando il comando ROS2.

```
cd /home/serl/GitHub/STRESA/ROS/src
ros2 pkg create --build-type ament_python nome_package
```

- **Secondo metodo:** Creazione del pacchetto copiando e rinominando un pacchetto esistente.

1. Copiare e rinominare un pacchetto esistente, incluso il nome della cartella contenente i file `driver.py` e `human_driver.py`.
2. Modificare i seguenti file per aggiornare il nome del pacchetto:

– **setup.py:**

```
package_name='nome_package_nuovo'
'driver=nome_package_nuovo:main',
'human_driver=nome_package_nuovo:main'
```

– **setup.cfg:**

```
script_dir=$base/lib/nome_package_nuovo
install_scripts=$base/lib/nome_package_nuovo
```

– **package.xml:**

```
<name>nome_package_nuovo</name>
```

– **resource:** Per i file .urdf, aggiornare il plugin come segue:

- * Per i file relativi a slam e localization:

```
<plugin type="nome_package_nuovo.driver.TIAGODriver">
```

- * Per il file relativo a human:

```
<plugin type="nome_package_nuovo.driver.Pedestrian">
```

– **launch:** Nei file .py, aggiornare le seguenti variabili:

- * **robot_description:** Variabile che carica il file .urdf del robot come stringa.
- * **human_description:** Variabile che carica il file .urdf dell'umano come stringa.
- * **webots:** Specifica il file .wbt della simulazione.

Successivamente fare ci che è descritto in 1.1.

1.4 Creazione di un nuovo Mondo di simulazione

- **Primo metodo:** Creare un nuovo scenario di simulazione in Webots. Per i robot per cui si desidera sviluppare un driver all'interno del nodo Webots, è necessario:
 - Impostare l'attributo *supervisor* su *true*.
 - Configurare il *controller* su *extern*.

Successivamente, creare un file .urdf per il nuovo robot. Come riferimento, utilizzare i file descritti in 1.3.

- **Secondo metodo:** Modificare uno scenario di simulazione esistente e salvare il mondo aggiornato nella sotto-cartella **worlds**. Procedere quindi con l'aggiornamento dei file rilevanti, seguendo le indicazioni fornite in 1.3.

Dopo aver configurato il nuovo scenario, seguire i passaggi descritti in 1.1. Per maggiori dettagli, consultare la tesi.

1.5 Metodo alternativo per apportare modifiche

Per modifiche minori, come l'aggiornamento di alcuni parametri, è possibile accedere direttamente al file `.wbt` e individuare il valore del campo da modificare. Dopo aver effettuato le modifiche, è importante seguire i passaggi descritti in 1.1.

1.6 Selezione del laser

All'interno dei file `.urdf` di slam e localizzazione modificare il contenuto dentro tag `laser` con `slamtec` oppure `urg`.

1.7 Util

Qui dentro sono contenute le classi che utilizzo all'interno del driver del robot.

- **ClassMarker:** Classe per i marke.
- **ClassObject:** Classe per i marker ed altri oggetti da riconoscere.
- **LaserObject:** Classe per il setup dei laser.
- **RoverInformation:** Classe per le informazioni relative al robot, sono già definiti dei valori di default.

1.8 aruco_msgs

Package contenente la struttura dei messaggi di aruco con estensione `.msg`.

1.9 aurora_msgs

Package contenente la struttura dei messaggi per il riconoscimento degli oggetti con estensione `.msg`.

1.10 rover_msgs

Package contenente la struttura dei messaggi per la descrizione delle informazioni del robot con estensione **.idl**.

1.11 webots_hospital_tiago

Package contente il codice relativo al driver del robot e quello per l'ambiente.

- Driver del robot Funzioni principali:
 - **init**: Funzione dove vengono inizializzati i parametri di simulazione
 - **step**: Funzione che viene eseguita ad ogni ciclo di simulazione

Funzioni secondarie:

- **init_components**: Inizializza la velocità delle ruote e il time step della simulazione.
- **init_tf_odom**: Configura il broadcaster statico per la trasformazione odom → base_link.
- **init_laser**: Configura il laser basandosi sui parametri definiti nel file `.urdf`.
- **setup_robot_devices**: Inizializza i dispositivi del robot in simulazione, come motori e sensori.
- **setup_publishers_and_subscribers**: Configura i publisher e i subscriber per lo scambio di messaggi.
- **init_messages**: Inizializza i messaggi scambiati tra i publisher e i subscriber.
- **setup_laser_msg**: Configura il messaggio LaserScan basandosi sul tipo di laser definito nel file `.urdf`.
- **setup_rover_msg**: Configura il messaggio RoverInfo con i valori predefiniti nella classe `RoverInformation`.

-
- **init_info_camera:** Inizializza matrici di rotazione, rototraslazione, quaternioni e velocità della struttura cinematica (base → visual → pan → tilt → camera).
 - **init_tf_camera:** Configura il broadcaster statico per la trasformazione base → visual → pan → tilt → camera.
 - **take_data_from_Webots_node:** Itera sui nodi Webots per accedere ai parametri di simulazione e configurare i campi necessari.
 - **__cmd_vel_callback:** Callback per la gestione dei comandi Twist del robot.
 - **__camera_callback:** Callback per il controllo della telecamera.

Altre funzioni:

- **quaternion_to_axis_angle:** Funzione per convertire un quaternione nella rappresentazione asse-angolo.
- **quat2eulers:** Funzione per convertire un quaternione negli angoli di Eulero.
- **build_matrix:** Funzione che converte un vettore 1×16 in una matrice 4×4 .
- **build_matrix_from_R:** Funzione per costruire una matrice di roto-traslazione 4×4 a partire da una matrice di rotazione 3×3 .

- Driver Ambiente

Funzioni primarie:

- **init:** Funzione che inizializza le variabili necessarie per la simulazione della camminata dell'umano, la velocità dell'umano e dell'oggetto. Accede ai parametri di simulazione, come le intensità delle luci, tramite le proprietà di Webots, salvandoli in apposite variabili. Inoltre, configura i subscriber per consentire la variazione dinamica dell'ambiente nella simulazione.
- **step:** Funzione eseguita ad ogni ciclo di simulazione per aggiornare lo stato del sistema.

Funzioni secondarie:

- **__cmd_vel_callback**: Funzione di callback per gestire il movimento dell'uomo nella simulazione.
- **__cmd_vel_object_callback**: Funzione di callback per gestire il movimento dell'oggetto nella simulazione.
- **__illuminance_callback**: Funzione di callback per impostare e regolare i parametri di illuminazione.

Altre funzioni: identiche a quelle del driver per il robot.

2 Descrizione funzioni Driver Webots Tiago

2.1 init components

Come si può capire dall'immagine 1 sono inizializzate le variabili *forward_speed*, *angular_speed*, *command_motor_left*, *command_motor_right* per il controllo delle velocità del robot Tiago. Le variabili *count_time_stamp_scan*, *count_time_stamp_odometry*, *count_time_stamp_marker* sono inizializzate qui e fanno riferimento al periodo della pubblicazione dei relativi messaggi.

```
# FUNZIONI DI SETUP
''' FUNZIONI DI SETUP DENTRO INIT()'''
def init_components(self):

    self.forward_speed = 0.0
    self.angular_speed = 0.0
    self.command_motor_left = 0.0
    self.command_motor_right = 0.0
    self.count_time_stamp_scan = 0.0
    self.count_time_stamp_odometry = 0.0
    self.count_time_stamp_marker = 0.0
```

Figura 1: Funzione init_component

2.2 init_tf_odom

Come si può vedere dall'immagine 2 sono inizializzate le variabili per la pubblicazione dei **tf** statici e dinamici che sono *static_broadcaster* e *broadcaster*.

Inoltre è inizializzata la variabile *odom_transform* che rappresente il messaggio di tipo *TransformStamped* per la l'aggiornamento e la pubblicazione del frame *odom* to *base_link*.

```
# INIZIALIZZAZIONE TF
def init_tf_odom(self):

    self.static_broadcaster = StaticTransformBroadcaster(self.__node)
    self.broadcaster = TransformBroadcaster(self.__node)
    self.odom_trasfrom = TransformStamped()
    self.odom_trasfrom.header.frame_id = 'odom'
    self.odom_trasfrom.child_frame_id = 'base_link'
```

Figura 2: Funzione init_tf_odom

2.3 init_laser

Come si può vedere dall'immagine 3 sono inizializzate le variabili per *laser_name*, *laser_x*, *laser_y*, *laser_x* per la descrizione frame e delle posizioni iniziali del laser. La variabile *tipo_laser* è una variabile che tramite il metodo *properties.get('laser')* legge il conenuto del tag *laser* contenuto all'interno del file *.urdf* e lo salva nella variabile destinata. Successivamente si presentano 3 basi basati sul valore contenuto dalla variabile *laser_name*:

- caso in cui *laser_name = urg*: Sempre tramite il metodo *properties.get* vengono assegnati dei valori alle variabili *laser_x*, *laser_y*, *laser_z*. Successivamente viene stampato a video il valore letto.
- caso in cui *laser_name = slamtec*:

Sempre tramite il metodo *properties.get* vengono assegnati dei valori alle variabili *laser_x*, *laser_y*, *laser_z*. Successivamente viene stampato a video il valore letto.

- caso in cui c'è scritto altro: Non viene aggiornato nessun valore e si utilizzano i valori di default.

```
# SCELTA DEL LASER DA UTILIZZARE DA FILE DI CONFIGURAZIONE
def init_laser(self, properties):
    """Configura il laser (URG o SLAMTEC) in base alle proprietà."""

    self.laser_name = 'laser_link'
    # POSIZIONE DEL LASER
    self.laser_x = 0.0
    self.laser_y = 0.0
    self.laser_z = 0.0

    tipo_laser = properties.get('laser')

    if tipo_laser == 'urg':
        self.__node.get_logger().info('Laser selezionato: URG')
        self.laser_name = 'laser_urg'

        self.laser_x = float(properties.get('laser_urg_x'))
        self.laser_y = float(properties.get('laser_urg_y'))
        self.laser_z = float(properties.get('laser_urg_z'))

        self.__node.get_logger().info('LASER_URG_X' + str(self.laser_x))
        self.__node.get_logger().info('LASER_URG_Y' + str(self.laser_y))
        self.__node.get_logger().info('LASER_URG_Z' + str(self.laser_z))
```

Figura 3: Funzione init_laser

```
elif tipo_laser == 'slamtec':
    self.__node.get_logger().info('Laser selezionato: SLAMTEC')
    self.laser_name = 'laser_slamtec'

    self.laser_x = float(properties.get('laser_slamtec_x'))
    self.laser_y = float(properties.get('laser_slamtec_y'))
    self.laser_z = float(properties.get('laser_slamtec_z'))

    self.__node.get_logger().info('LASER_SLAMTEC_X' + str(self.laser_x))
    self.__node.get_logger().info('LASER_SLAMTEC_Y' + str(self.laser_y))
    self.__node.get_logger().info('LASER_SLAMTEC_Z' + str(self.laser_z))

else:
    self.__node.get_logger().info('Tipo di laser non specificato.')
    return
```

Figura 4: Funzione init_laser

2.4 setup_robot_devices

Come si può vedere dall'immagine 5 tramite il metodo `getDevice()` (**nome_device**) vengono assegnati alle variabili `__motor_left`, `__motor_right`, `__sensor_left`, `__sensor_right`, `__lidar`, `gps`, `imu` e `__camera` i riferimenti ad oggetti presenti in simulazione.

```
# SETUP DEVICES IN SIMULAZIONE
def setup_robot_devices(self, properties):

    """Configura i dispositivi del robot (motori, sensori, lidar, ecc.)."""
    self.__motor_left = self.__robot.getDevice('wheel_left_joint')
    self.__motor_right = self.__robot.getDevice('wheel_right_joint')
    self.__sensor_left = self.__robot.getDevice('wheel_left_joint_sensor')
    self.__sensor_right = self.__robot.getDevice('wheel_right_joint_sensor')
    self.__lidar = self.__robot.getDevice(self.laser_name)

    self.gps = self.__robot.getDevice('gps')
    self.imu = self.__robot.getDevice('inertial unit')
    self.__camera = self.__robot.getDevice('TiagoCamera')
```

Figura 5: Funzione `setup_robot_devices`

Successivamente come si può vedere nell'immagine 6 viene eseguito un ciclo `for` per abilitare il timestamp di simulazione anche ai dispositivi presenti in simulazione. Vengono poi settati i valori di posizione e velocità per 2 ruote del robot. Per lidar e telecamera si sono abilitati anche i *pointcloud* e il riconoscimento degli oggetti.

```
for device in [self.__sensor_left, self.__sensor_right, self.gps, self.imu]:
    device.enable(self.__timestep)

self.__motor_left.setPosition(float('inf'))
self.__motor_right.setPosition(float('inf'))
self.__motor_left.setVelocity(0.0)
self.__motor_right.setVelocity(0.0)

self.__lidar.enable(self.__timestep)
self.__lidar.enablePointCloud()
self.__camera.enable(self.__timestep)
self.__camera.recognitionEnable(self.__timestep)
```

Figura 6: Funzione `setup_robot_devices`

2.5 setup_publishers_and_subscribers

Qui semplicemente vengono inizializzati i publishers ed i subscribers.

- *odometry_publisher*: ha il topic **odom** e **Odometry** come tipo di messaggio.
- *laser_publisher*: ha il topic **scan_webots** e **LaserScan** come tipo di messaggio.
- *rover_info_publisher* ha il topic **rover_info** e **RoverInfo** come tipo di messaggio.
- *objects_publisher* ha il topic **visual_objects** e **Object3DArray** come tipo di messaggio.
- *il primo subscriber* ha il topic **rover_twist** e **Twist** come tipo di messaggio.
- *il secondo subscriber* ha il topic **camera_vel** e **JointState** come tipo di messaggio.

La parte commentata è un esempio di come posso cambiare le *QoS* di un publisher o di un subscriber per renderlo compatibile con componenti esterni nel caso servisse.

```
# SETUP PUBLISHER E SUBSCRIBERS
def setup_publishers_and_subscribers(self):
    """Crea i publisher e i subscriber ROS necessari."""

    #***** SET QoS for STRESA connection*****
    # qos_profile = QoSProfile(
    #     reliability=QoSReliabilityPolicy.RELIABLE,
    #     history=QoSHistoryPolicy.KEEP_LAST,
    #     durability=QoSDurabilityPolicy.TRANSIENT_LOCAL,
    #     depth=10
    # )
    #*****

    self.odometry_publisher = self.__node.create_publisher(Odometry, 'odom', 1)
    self.laser_publisher = self.__node.create_publisher(LaserScan, 'scan_webots', 1)
    self.rover_info_publisher = self.__node.create_publisher(RoverInfo, 'rover_info', 1)
    self.objects_publisher = self.__node.create_publisher(Object3DArray, 'visual_objects', 1)

    self.__node.create_subscription(Twist, 'rover_twist', self.__cmd_vel_callback, 1)
    # self.__node.create_subscription(Twist, 'camera_vel_sim', self.__camera_callback, 1)
    self.__node.create_subscription(JointState, 'camera_vel', self.__camera_callback, 1)
```

Figura 7: Funzione setup_publishers_and_subscribers

2.6 init_messages

Come si può vedere dall'immagine 8 sono inizializzati le variabili per la pubblicazione o la ricezione dei messaggi.

- La variabile *target_twist* è un messaggio di tipo **Twist**.
- La variabile *messaggio_object* è un messaggio di tipo **Object3DArray**.
- La variabile *messaggio_odometria* è un messaggio di tipo **Odometry**.
- La variabile *messaggio_webots_scanner* è un messaggio di tipo **LaserScan**.
- La variabile *messaggio_rover_info* è un messaggio di tipo **RoverInfo**.
- La variabile *pan_twist* è un messaggio di tipo **Twist**.
- La variabile *tilt_twist* è un messaggio di tipo **Twist**.

Per il *pan_twist* e *tilt_twist* non sono utilizzati perché sono stati sostituiti dal tipo di messaggio *JointState*.

```
# DEFINIZIONE MESSAGGI ROS
def init_messages(self):
    """Inizializza i messaggi ROS."""
    self.target_twist = Twist()
    self.messaggio_objects = Object3DArray()
    self.messaggio_odometria = Odometry()
    self.messaggio_webots_scanner = LaserScan()
    self.messaggio_rover_info = RoverInfo()

    self.pan_twist = Twist()
    self.tilt_twist = Twist()
```

Figura 8: Funzione init_messages

2.7 setup_laser_msg

Nell'immagine 9 viene istanziato l'oggetto di tipo *LaserObject* la cui classe è descritta all'interno del folder *util*. La variabile *laser_object* viene inizializzato tramite il metodo *setup* che richiede i seguenti argomenti:

- modello
- max range
- min range
- risoluzione
- campo visivo

```
# SETUP MESSAGGIO LASER
def setup_laser_msg(self):
    '''LASERSCAN MESSAGE'''
    self.laser_object = LaserObject.LaserObject()
    self.laser_object.setup('laser_link',
                            self.__lidar.getModel(),
                            self.__lidar.getMaxRange(),
                            self.__lidar.getMinRange(),
                            self.__lidar.getHorizontalResolution(),
                            self.__lidar.getFov()
                            )
```

Figura 9: Funzione setup_laser_msg

Nell'immagine 10 viene riempito il messaggio con le informazioni relative alla scansione laser.

```
self.messaggio_webots_scanner.header.frame_id = "laser_link"
self.messaggio_webots_scanner.angle_max = self.laser_object.angle_max
self.messaggio_webots_scanner.angle_min = self.laser_object.angle_min

self.messaggio_webots_scanner.range_max = self.laser_object.range_max
self.messaggio_webots_scanner.range_min = self.laser_object.range_min
# messaggio_webots_scanner.scan_time = scanner.scan_time
self.messaggio_webots_scanner.time_increment = 0.0
self.messaggio_webots_scanner.scan_time = 0.0032
self.messaggio_webots_scanner.angle_increment = self.laser_object.angle_increment
```

Figura 10: Funzione setup_laser_msg

2.8 setup_rover_msg

Nell'immagine 11 viene istanziato l'oggetto di tipo *RoverInformation* la cui classe è descritta all'interno del folder *util*. Viene istanzata con i valori di default che sono all'interno del costruttore della classe stessa. Successivamente viene riempito il messaggio *messaggio_rover_info*.

```
# SETUP ROVERS MESSAGE
def setup_rover_msg(self):

    self.rover_info = RoverInformation.RoverInformation()

    self.messaggio_rover_info.footprint[0] = self.rover_info.min_x
    self.messaggio_rover_info.footprint[1] = self.rover_info.max_x
    self.messaggio_rover_info.footprint[2] = self.rover_info.min_y
    self.messaggio_rover_info.footprint[3] = self.rover_info.max_y
    self.messaggio_rover_info.footprint[4] = self.rover_info.min_z
    self.messaggio_rover_info.footprint[5] = self.rover_info.max_z

    self.messaggio_rover_info.safety_margin = self.rover_info.safety_margin
    self.messaggio_rover_info.min_lin_vel = self.rover_info.min_lin_vel
    self.messaggio_rover_info.max_lin_vel = self.rover_info.max_lin_vel
    self.messaggio_rover_info.min_ang_vel = self.rover_info.min_ang_vel
    self.messaggio_rover_info.max_ang_vel = self.rover_info.max_ang_vel
    self.messaggio_rover_info.max_lin_acc = self.rover_info.max_lin_acc
    self.messaggio_rover_info.max_ang_acc = self.rover_info.max_ang_acc
    self.messaggio_rover_info.max_cen_acc = self.rover_info.max_cen_acc
    self.messaggio_rover_info.lin_resolution = self.rover_info.lin_resolution
    self.messaggio_rover_info.ang_resolution = self.rover_info.ang_resolution
```

Figura 11: Funzione setup_rover_msg

2.9 init_info_camera

```
def init_info_camera(self):  
    # VETTORE PER IL CONTROLLO DEL  
  
    self.R_visual_link = []  
    self.M_visual_link = []  
    self.quaternion_visual = []  
  
    self.R_pan_odom = []  
    self.M_pan_odom = []  
  
    self.visual_link = []  
    self.pan_link = []  
  
    self.R_pan_tilt_link = []  
    self.M_pan_tilt_link = []  
    self.pan_tilt_link = []  
    self.quaternion_pan_tilt = []  
  
    self.R_tilt_odom = []  
    self.M_tilt_odom = []  
    self.tilt_link = []  
    self.tilt_link_odom = []  
    self.quaternion_tilt_odom = []  
  
    self.R_camera_link = []  
    self.M_camera_link = []  
  
    self.R_tiago_M = []  
    self.M_tiago_M = []  
    self.M_tiago_finale_M = []  
    self.quaternion_camera = []  
  
    self.camera_link_odom = []  
    self.camera_link = []  
    self.tiago_M = []  
  
    self.pan_speed = 0.0  
    self.tilt_speed = 0.0
```

Figura 12: Funzione init_tf_camera

Nell'immagine 12 vengono inizializzate le variabili per lo svolgimento delle operazioni delle roto_traslazioni e quelle relative alla velocità del *pan* e del *tilt*.

2.10 init_tf_camera

Nell'immagine 13 viene riempito il *tf* relativo al *base_link* to *visual_link*. Successivamente viene pubblicato dal relativo *broadcaster*.

```
# ***** VISUAL_TILT_LINK *****

self.TF_visual_link = TransformStamped()
self.TF_visual_link.header.stamp = self.__node.get_clock().now().to_msg()
self.TF_visual_link.header.frame_id = 'base_link'
self.TF_visual_link.child_frame_id = 'visual_link'

self.TF_visual_link.transform.rotation.w = self.quaternion_visual[3]
self.TF_visual_link.transform.rotation.x = self.quaternion_visual[4]
self.TF_visual_link.transform.rotation.y = self.quaternion_visual[5]
self.TF_visual_link.transform.rotation.z = self.quaternion_visual[6]
self.TF_visual_link.transform.translation.x = 0.14
self.TF_visual_link.transform.translation.y = 0.0
self.TF_visual_link.transform.translation.z = 0.343

self.static_broadcaster.sendTransform(self.TF_visual_link) # VALORI COSTANTI
```

Figura 13: Funzione init_tf_camera

Nell'immagine 14 vengono effettuate le relative conversioni tra da axis angle a matrici di rotazione, roto-traslazione e quaternioni.

```
# VALORI DA SETTARE

self.TF_pan_2_tilt.transform.rotation.w = quaternion_tilt[3]
self.TF_pan_2_tilt.transform.rotation.x = quaternion_tilt[4]
self.TF_pan_2_tilt.transform.rotation.y = quaternion_tilt[5]
self.TF_pan_2_tilt.transform.rotation.z = quaternion_tilt[6]

self.TF_pan_2_tilt.transform.translation.x = 0.0
self.TF_pan_2_tilt.transform.translation.y = 0.0
self.TF_pan_2_tilt.transform.translation.z = 0.0
```

Figura 14: Funzione init_tf_camera

Nell'immagine 15 vengono inizializzati i messaggi relativi ai *tf* *visual_link* to *pan_link* e *pan_link* to *tilt_link*.

```
# ***** PAN LINK TILT LINK *****
self.TF_pan_link = TransformStamped()
self.TF_pan_link.header.stamp = self.__node.get_clock().now().to_msg()
self.TF_pan_link.header.frame_id = 'visual_link'
self.TF_pan_link.child_frame_id = 'pan_link'

# ***** PAN_LINK   TILT_LINK *****          ANGOLO VARIABILE

self.TF_pan_2_tilt = TransformStamped()
self.TF_pan_2_tilt.header.stamp = self.__node.get_clock().now().to_msg()
self.TF_pan_2_tilt.header.frame_id = 'pan_link'
self.TF_pan_2_tilt.child_frame_id = 'tilt_link'
```

Figura 15: Funzione init_tf_camera

Nell'immagine 16 vengono riempito il messaggio relativo alla roto-traslazione tra *pan* e *tilt*

```
#   VALORI DA SETTARE

self.TF_pan_2_tilt.transform.rotation.w = quaternion_tilt[3]
self.TF_pan_2_tilt.transform.rotation.x = quaternion_tilt[4]
self.TF_pan_2_tilt.transform.rotation.y = quaternion_tilt[5]
self.TF_pan_2_tilt.transform.rotation.z = quaternion_tilt[6]

self.TF_pan_2_tilt.transform.translation.x = 0.0
self.TF_pan_2_tilt.transform.translation.y = 0.0
self.TF_pan_2_tilt.transform.translation.z = 0.0
```

Figura 16: Funzione init_tf_camera

Nell'immagine 17 vengono riempito il messaggio relativo alla roto-traslazione tra *tilt* e *camera_link*

```

***** TILT_LINK CAMERA_LINK FISSO *****

self.TF_camera_link = TransformStamped()
self.TF_camera_link.header.stamp = self.__node.get_clock().now().to_msg()
self.TF_camera_link.header.frame_id = 'tilt_link'
self.TF_camera_link.child_frame_id = 'camera_link'

self.TF_camera_link.header.stamp = self.__node.get_clock().now().to_msg()
self.TF_camera_link.transform.rotation.w = self.quaternion_camera[3]
self.TF_camera_link.transform.rotation.x = self.quaternion_camera[4]
self.TF_camera_link.transform.rotation.y = self.quaternion_camera[5]
self.TF_camera_link.transform.rotation.z = self.quaternion_camera[6]
self.TF_camera_link.transform.translation.x = 0.0
self.TF_camera_link.transform.translation.y = 0.0
self.TF_camera_link.transform.translation.z = 0.0

self.static_broadcaster.sendTransform(self.TF_camera_link)

```

Figura 17: Funzione `init_tf_camera`

2.11 `take_data_from_Webots_node`

Parte di codice molto corposa i metodi principali sono:

- `root_children_field.getMFNode(k)`: tramite il quale accedo al k-esimo nodo di `webots`.
- `getTypeName()`: tramite il quale ricerco il nome del nodo.
- `getField`: tramite il quale prendo il valore all'interno del campo definito.

2.12 `__cmd_vel_callback`

Nell'immagine 18 viene letto il messaggio ricevuto nel topic *Twist* e viene salvato nelle variabili *forward_speed* e *angular_speed*.

```
# TWIST CALLBACK
def __cmd_vel_callback(self, twist): # Idea cambiare i valori ad ogni chiamata d

    self.forward_speed = twist.linear.x
    self.angular_speed = twist.angular.z
```

Figura 18: Funzione `__cmd_vel_callback`

2.13 `__cmd_vel_callback`

Nell'immagine 19 viene letto il messaggio ricevuto nel topic *camera_vel* e viene salvato nelle variabili *pan_speed* e *tilt_speed*.

```
def __camera_callback(self, jointstate):
    # joint_msg.velocity()[1] = tilt_speed;
    self.pan_speed = jointstate.velocity[0]

    self.tilt_speed = jointstate.velocity[1]
```

Figura 19: Funzione `__camera_callback`

2.14 Funzione *step*

La funzione *step* è la funzione che viene eseguito ad ogni ciclo di simulazione. Il codice è molto lungo ma è commentato bene nel caso in cui si vogliano effettuare delle modifiche.